

Rezolvări detaliate - subiecte PAOJ

Grile, stream-uri, fișiere, fire de executare și clase imutabile

Cuprins

Cum să folosești PDF-ul	2
1 Examen PAO 2020-2021 - grile	3
2 Subiect 01 - 2022	8
2.1 Partea I - grile	8
2.2 Partea II - clasa Imobil și stream-uri	9
2.3 Partea III - fișiere + fir de executare pentru imobile	10
2.4 Partea IV - clasă imutabilă Firma	11
3 Colocviu 2023 - 2022 - 2023.pdf	13
3.1 Partea I - grile	13
3.2 Partea II - clasa Produs și stream-uri	14
3.3 Partea III - fișiere + fire pentru produse	15
3.4 Partea IV - clasă imutabilă Facultate	16
4 Subiect 01 - 2024, seria 25	18
4.1 Partea I - grile	18
4.2 Partea II - clasa TennisATP și stream-uri	19
4.3 Partea III - fișiere + fire pentru jucători	20
4.4 Partea IV - Facultate imutabilă	21
5 Subiect 02 - 2024, seriile 23 și 24	22
5.1 Partea I - grile	22
5.2 Partea II - Imobil1 și stream-uri	23
5.3 Partea III - fișiere + fire pentru AgentieRS	24
5.4 Partea IV - singleton CitireInformatiiPersoane	25
6 Examen 2024-2025	27
6.1 Partea I - grile	27
6.2 Partea II - clasa Imobil și stream-uri	28
6.3 Partea III - fișiere + fire pentru localitate	29
6.4 Partea IV - clasă imutabilă Imobil cu StringBuilder	30
7 Tabel rapid de reguli pentru examen	32

Cum să folosești PDF-ul

Documentul este scris ca pentru recapitulare de examen: la grile ai răspunsul și regula care justifică răspunsul, iar la exercițiile de cod ai variante complete, ușor de adaptat pe foaie.

Regulă de examen

La examen, când vezi `HashSet`, `HashMap`, `equals()` și `hashCode()`, gândește în ordinea aceasta:

1. se calculează bucket-ul pe baza lui `hashCode()`;
2. doar în același bucket se verifică `equals()`;
3. dacă `equals()` spune că două chei sunt egale, în `HashMap` se înlocuiește valoarea, iar în `HashSet` nu se mai adaugă elementul.

Dacă `equals()` și `hashCode()` sunt scrise greșit, programul tot rulează, dar rezultatul poate părea contraintuitiv.

1 Examen PAO 2020-2021 - grile

Întrebarea 3 - String, substring, intern, equals

Răspuns pe scurt

Adevărate:

- `text.endsWith(text.substring(5))`
- `text.toLowerCase().intern()==text.toLowerCase().intern()`
- `text.toLowerCase().equals(text.toLowerCase())`

Explicație.

- `text.endsWith(text.substring(5))` este adevărată: `substring(5)` este chiar sufixul șirului începând de la indexul 5, iar orice șir se termină cu propriul său sufix.
- `intern()` pune șirul în string pool. Două rezultate identice internate vor avea aceeași referință, deci `==` devine adevărat.
- `equals()` compară valoarea, deci două apeluri `toLowerCase()` cu același conținut sunt egale.
- `text.replaceAll("a", "").contains("a")` este falsă pentru litera mică a, deoarece toate aparițiile lui a au fost eliminate. Dacă textul conține doar A, acestea rămân, dar `contains("a")` caută litera mică.

Întrebarea 4 - overriding, equals, hiding, super()

Răspuns pe scurt

Adevărate: metoda suprascrisă poate întoarce un tip mai derivat; constructorul superclasei se poate apela doar la începutul constructorului clasei derivate.

Explicație.

- Tipul returnat poate fi covariant. Exemplu: în superclasă metoda întoarce A, iar în subclasă poate întoarce B `extends A`.
- `Object.equals()` nu compară câmp cu câmp. Implicit compară identitatea obiectelor, adică aproximativ ca `==`.
- O dată membră ascunsă printr-un câmp cu același nume în subclasă nu devine inaccesibilă: se poate accesa din subclasă cu `super.numeCamp`.
- Apelul `super(...)` din constructor trebuie să fie prima instrucțiune.

Întrebarea 5 - fișiere A.java, B.java, C.java

Răspuns pe scurt

Conțin erori: varianta cu `class C extends A,B` și varianta cu `public static class B` ca top-level class.

Explicație. Java nu permite moștenire multiplă de clase, iar o clasă de nivel superior nu poate fi declarată static. `static class` este permisă doar ca nested class.

Întrebarea 6 - fire de executare recursive

Răspuns pe scurt

Valoarea 9999999 apare de **9 ori**, presupunând că programul s-ar termina.

Ideea. Fie $f(n)$ numărul de fire care ajung să afișeze nivelul n . Pentru $n < 2$, fiecare fir creează 3 fire de nivel $n + 1$. Pentru $n \geq 2$, fiecare fir creează doar un fir de nivel $n + 1$.

$$f(0) = 1, \quad f(1) = 3, \quad f(2) = 9, \quad f(3) = 9, \dots, f(9999999) = 9.$$

Întrebarea 7 - try-with-resources, catch, finally

Răspuns pe scurt

Output: ?F2F?F4F!F, apoi programul se termină cu excepție.

Explicație pe pași.

- file1.txt este gol: nextInt() aruncă NoSuchElementException; se afișează ?, apoi finally afișează F.
- file2.txt: se citește 2, apoi F.
- file3.txt: gol, deci ?F.
- file4.txt: se citește 4, apoi F.
- file5.txt: nu există, se afișează !, se relansează excepția cu throw e, apoi se execută finally, deci F. Programul se termină cu excepție.

Întrebarea 8 - fire de executare

Răspuns pe scurt

Adevărate: firele au stivă proprie și heap partajat; un fir aflat în wait() poate fi trezit prin notify() apelat pe același obiect.

Explicație. Firul pornește cu start(), nu prin apel direct la run(). Un fir se poate defini fie prin extinderea clasei Thread, fie prin implementarea interfeței Runnable.

Întrebarea 9 - colecții

Răspuns pe scurt

Adevărată: folosind add() în LinkedHashSet și ArrayList se reține ordinea inserării elementelor.

Explicație. Collection extinde Iterable, deci colecțiile sunt iterabile. TreeSet este o colecție iterabilă. Suprascrierea lui hashCode() și equals() nu garantează absența coliziunilor; coliziuni pot exista oricum.

Întrebarea 10 - HashMap cu equals mereu true

Răspuns pe scurt

În Java, tabela conține două intrări: una pentru hash-ul lui A(1) și una pentru hash-ul lui A(2).

Explicație. Codul încalcă regula importantă: dacă `a.equals(b)` este `true`, atunci trebuie să avem `a.hashCode() == b.hashCode()`. Totuși, implementarea `HashMap` caută cheia egală în bucket-ul dat de hash. A(1) are hash 1, iar A(2) are hash 2, deci ajung în bucket-uri diferite și pot coexista.

Întrebarea 11 - suprascriere, supraîncărcare, erori

Răspuns pe scurt

Răspunsul de grilă uzual: 2 suprascrieri, 1 supraîncărcare, 2 erori. Strict după JLS, metoda privată din superclasă nu este moștenită, deci analiza poate fi nuanțată.

Explicație pentru grilă.

- `met1()`: suprascriere validă cu tip returnat covariant, B în loc de A.
- `met2()`: în grilă este tratată ca suprascriere, deși în Java metodele private nu se moștenesc.
- `met3()`: eroare, deoarece reduce accesul de la public la private.
- `met4()`: eroare, deoarece metoda statică ascunsă nu poate avea acces mai restrictiv.
- `met5()`: supraîncărcare prin listă diferită de parametri.

Întrebarea 12 - lambda

Răspuns pe scurt

Adevărate: lambda expresiile pot avea lista parametrilor vidă; uneori se pot omite acoladele.

Explicație. O interfață funcțională are exact o metodă abstractă, nu „cel puțin una”. `Comparator` este folosită pentru comparare/sortare, nu pentru egalitate.

Întrebarea 13 - lambda și output

Răspuns pe scurt

Valoarea lui `x` este 18, deci afirmația adevărată este: `x` este multiplu de 3.

Calcul.

- `ob1.f(ob2.f())` afișează `obiect.get() * 5`. Outputul este 15, deci `obiect.get() = x / 5 = 3`. Rezultă $x \in \{15, 16, 17, 18, 19\}$.
- `obiect.toString()` este `"A" + (x % 5)`. După `substring(1)` se afișează restul `x%5`. Outputul este 3, deci $x \% 5 = 3$.
- Singura valoare este $x = 18$.

Întrebarea 14 - constructori și modificatori

Răspuns pe scurt

Adevărate: o dată membră publică poate fi accesată și din afara clasei; un constructor poate fi public sau private.

Atenție. Constructorii nu pot fi static, abstract sau final. Clasele abstracte pot fi extinse.

Întrebarea 15 - stream pe lista de persoane

Răspuns pe scurt

Rezultatul este un șir de caractere format prin concatenarea numelor sortate ale persoanelor care încep cu B și au salariu mai mare de 2000.

Pipeline-ul face: filter după nume, filter după salariu, map la nume, sorted, apoi joining(", ").

Întrebarea 16 - stream-uri

Răspuns pe scurt

Adevărate: după deschiderea unui stream se pot efectua mai multe operații intermediare și o singură operație terminală; Stream.iterate poate produce un șir de numere impare.

Operațiile intermediare sunt lazy: se aplică doar când apare operația terminală. Stream-urile nu sunt reutilizabile.

Întrebarea 17 - referințe către obiecte

Răspuns pe scurt

Regula corectă: ce metode poți apela este decis de tipul referinței; ce implementare se execută pentru metodele nestatice suprascrise este decis de tipul real al obiectului.

Pentru examen:

- referință de tipul clasei: ai acces la metodele publice ale clasei;
- referință de tip superclasă: ai acces la metodele declarate în superclasă; dacă sunt suprascrise, se execută varianta din subclasă;
- referință de tip interfață: ai acces la metodele declarate în interfață;
- referință de tip Object: ai acces doar la metodele din Object, de exemplu toString(), equals(), hashCode().

Întrebarea 18 - clase imutabile A, B, C

Răspuns pe scurt

Răspunsul de grilă probabil: B și C. Pentru implementare riguroasă, câmpurile ar trebui să fie `private final`; altfel accesul `package-private` poate strica imutabilitatea.

Explicație. A este mutabilă deoarece are setter. B face copie defensivă în constructorul cu A și întoarce copie în `getA()`. C întoarce copie prin `getB()`. Totuși, pentru cod de producție, câmpurile fără `private` nu sunt acceptabile la o clasă imutabilă.

Întrebarea 19 - excepții

Răspuns pe scurt

Adevărate: excepțiile la compilare sunt obligatoriu de tratat; dacă `ExceptieB extends ExceptieA`, `catch`-ul pentru `ExceptieB` trebuie pus înaintea celui pentru `ExceptieA`.

Erorile nu sunt obligatoriu de tratat. Excepțiile sunt obiecte, nu tipuri primitive.

Întrebarea 20 - suspendarea unui fir

Răspuns pe scurt

Adevărate: firul se suspendă/blochează când încearcă să intre într-o metodă `synchronized` deja executată pe același obiect; firul se suspendă când apelează `legal wait()`.

`notify()` nu suspendă firul care îl apelează; dimpotrivă, trezește un fir aflat în așteptare pe același monitor. Pentru `wait()`, firul trebuie să dețină monitorul obiectului, altfel apare `IllegalMonitorStateException`.

2 Subiect 01 - 2022

2.1 Partea I - grile

1. HashMap cu equals mereu false

Răspuns pe scurt

Răspuns: 1, 1, 2, 2.

`equals()` întoarce mereu false, deci două obiecte diferite nu sunt considerate egale nici măcar dacă au același hash. De aceea se păstrează toate cele patru inserări.

2. Serializare și constructor

Răspuns pe scurt

Mesajul Constructor se afișează o singură dată.

Constructorul se apelează doar la `new Persoana(...)`. La deserializare cu `readObject()` nu se apelează constructorul clasei serializate.

3. Excepții și finally

Răspuns pe scurt

Output: ABDEC.

Se pornește de la A. Metoda `A()` adaugă B. Metoda `B()` adaugă D, apoi `C()` aruncă excepție. `catch` din `B()` relansează excepția, dar înainte de ieșire se execută `finally`, care adaugă E. Excepția ajunge în `A()`, unde `catch` adaugă C. Rezultatul este A B D E C.

4. String imutabil și substring

Răspuns pe scurt

Răspuns: 11.

`s.toUpperCase()` nu modifică șirul, deoarece `String` este imutabil. Rezultatul nu este atribuit. Deci în "abracadabra" nu există litera mare B; atât `indexOf("B")` cât și `lastIndexOf("B")` sunt -1. Lungimea rămâne 11.

5. Overriding, overloading, erori

Răspuns pe scurt

Răspuns: o suprascriere, o supraîncărcare, trei erori.

- `met1()`: suprascriere validă, cu tip returnat covariant.
- `met2()`: eroare, metoda din A este final.
- `met3()`: eroare, accesul scade de la public la private.
- `met4()`: eroare, se reduce accesul unei metode statice ascunse.

- met5(): supraîncărcare, pentru că lista parametrilor diferă.

2.2 Partea II - clasa Imobil și stream-uri

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 class Imobil {
5     private String tip;
6     private String localitate;
7     private int nrCamere;
8     private double suprafata;
9     private double pret;
10
11     public Imobil(String tip, String localitate, int nrCamere,
12         double suprafata, double pret) {
13         this.tip = tip;
14         this.localitate = localitate;
15         this.nrCamere = nrCamere;
16         this.suprafata = suprafata;
17         this.pret = pret;
18     }
19
20     public String getTip() { return tip; }
21     public String getLocalitate() { return localitate; }
22     public int getNrCamere() { return nrCamere; }
23     public double getSuprafata() { return suprafata; }
24     public double getPret() { return pret; }
25
26     public void setTip(String tip) { this.tip = tip; }
27     public void setLocalitate(String localitate) { this.localitate = localitate; }
28     public void setNrCamere(int nrCamere) { this.nrCamere = nrCamere; }
29     public void setSuprafata(double suprafata) { this.suprafata = suprafata; }
30     public void setPret(double pret) { this.pret = pret; }
31
32     @Override
33     public String toString() {
34         return tip + ", " + localitate + ", " + nrCamere + " camere, "
35             + suprafata + " mp, " + pret + " RON";
36     }
37
38     @Override
39     public boolean equals(Object o) {
40         if (this == o) return true;
41         if (!(o instanceof Imobil other)) return false;
42         return nrCamere == other.nrCamere
43             && Double.compare(suprafata, other.suprafata) == 0
44             && Double.compare(pret, other.pret) == 0
45             && Objects.equals(tip, other.tip)
46             && Objects.equals(localitate, other.localitate);
47     }
48
49     @Override
50     public int hashCode() {
51         return Objects.hash(tip, localitate, nrCamere, suprafata, pret);
52     }
53 }
54
55 public class TestImobil2022 {
56     public static void main(String[] args) {
57         List<Imobil> imobile = new ArrayList<>(List.of(
58             new Imobil("apartament", "Bucuresti", 3, 72.5, 450000),
59             new Imobil("garsoniera", "Cluj", 1, 35.0, 90000),
60             new Imobil("casa", "Bucuresti", 4, 120.0, 700000),
61             new Imobil("apartament", "Iasi", 3, 65.0, 95000)
62         ));

```

```

63
64 // a) cel puțin 3 camere, pret <= 100000, sortate crescator dupa suprafata
65 imobile.stream()
66     .filter(i -> i.getNrCamere() >= 3)
67     .filter(i -> i.getPret() <= 100000)
68     .sorted(Comparator.comparingDouble(Imobil::getSuprafata))
69     .forEach(System.out::println);
70
71 // b) localitati distincte
72 imobile.stream()
73     .map(Imobil::getLocalitate)
74     .distinct()
75     .forEach(System.out::println);
76
77 // c) colectie cu imobile din Bucuresti cu pret intre 200000 si 500000
78 List<Imobil> bucuresti = imobile.stream()
79     .filter(i -> i.getLocalitate().equalsIgnoreCase("Bucuresti"))
80     .filter(i -> i.getPret() >= 200000 && i.getPret() <= 500000)
81     .collect(Collectors.toList());
82
83 // d) pentru fiecare localitate distincta, lista imobilelor
84 imobile.stream()
85     .collect(Collectors.groupingBy(Imobil::getLocalitate))
86     .forEach((loc, lista) -> System.out.println(loc + " -> " + lista));
87 }
88 }

```

2.3 Partea III - fișiere + fir de executare pentru imobile

Cerință: din fișiere agentieRentSale_1.txt și agentieRentSale_2.txt, linii de forma tip,localitate,nrCamere,suprafata,pret, se citește smin și cmin, apoi se afișează numărul total de imobile cu suprafata > smin și nrCamere >= cmin.

```

1 import java.io.*;
2 import java.nio.file.*;
3 import java.util.*;
4
5 class ImobilParser2022 {
6     static Imobil parse(String linie) {
7         String[] v = linie.split(",");
8         return new Imobil(
9             v[0].trim(),
10            v[1].trim(),
11            Integer.parseInt(v[2].trim()),
12            Double.parseDouble(v[3].trim()),
13            Double.parseDouble(v[4].trim())
14        );
15    }
16 }
17
18 class NumaratorImobile extends Thread {
19     private final Path fisier;
20     private final double smin;
21     private final int cmin;
22     private int rezultat;
23
24     public NumaratorImobile(String fisier, double smin, int cmin) {
25         this.fisier = Path.of(fisier);
26         this.smin = smin;
27         this.cmin = cmin;
28     }
29
30     public int getRezultat() {
31         return rezultat;
32     }
33 }

```

```

33
34 @Override
35 public void run() {
36     try {
37         rezultat = (int) Files.lines(fisier)
38             .map(ImobilParser2022::parse)
39             .filter(i -> i.getSuprafata() > smin)
40             .filter(i -> i.getNrCamere() >= cmin)
41             .count();
42     } catch (IOException e) {
43         throw new RuntimeException(e);
44     }
45 }
46 }
47
48 public class MainImobileFisiere2022 {
49     public static void main(String[] args) throws Exception {
50         Scanner sc = new Scanner(System.in);
51         double smin = sc.nextDouble();
52         int cmin = sc.nextInt();
53
54         NumaratorImobile f1 = new NumaratorImobile("agentieRentSale_1.txt", smin, cmin);
55         NumaratorImobile f2 = new NumaratorImobile("agentieRentSale_2.txt", smin, cmin);
56
57         f1.start();
58         f2.start();
59         f1.join();
60         f2.join();
61
62         System.out.println(f1.getRezultat() + f2.getRezultat());
63     }
64 }

```

2.4 Partea IV - clasă imutabilă Firma

Model de răspuns. Pentru o clasă imutabilă:

- clasa se declară final;
- câmpurile sunt private final;
- nu există setteri;
- pentru obiecte mutabile, constructorul și getterul fac copii defensive.

```

1 import java.util.*;
2
3 class Adresa {
4     private String strada, numar, bloc, scara, etaj, apartament, judet, localitate;
5
6     public Adresa(String strada, String numar, String bloc, String scara,
7         String etaj, String apartament, String judet, String localitate) {
8         this.strada = strada;
9         this.numar = numar;
10        this.bloc = bloc;
11        this.scara = scara;
12        this.etaj = etaj;
13        this.apartament = apartament;
14        this.judet = judet;
15        this.localitate = localitate;
16    }
17
18    public Adresa(Adresa other) {
19        this(other.strada, other.numar, other.bloc, other.scara,
20            other.etaj, other.apartament, other.judet, other.localitate);

```

```
21     }
22 }
23
24 public final class Firma {
25     private final String denumireFirma;
26     private final int numarAngajati;
27     private final double profitMediuAnual;
28     private final Adresa adresa;
29
30     public Firma(String denumireFirma, int numarAngajati,
31                 double profitMediuAnual, Adresa adresa) {
32         this.denumireFirma = denumireFirma;
33         this.numarAngajati = numarAngajati;
34         this.profitMediuAnual = profitMediuAnual;
35         this.adresa = new Adresa(adresa); // copie defensiva
36     }
37
38     public String getDenumireFirma() { return denumireFirma; }
39     public int getNumarAngajati() { return numarAngajati; }
40     public double getProfitMediuAnual() { return profitMediuAnual; }
41
42     public Adresa getAdresa() {
43         return new Adresa(adresa); // nu expunem referinta interna
44     }
45 }
```

3 Colocviu 2023 - 2022 - 2023.pdf

3.1 Partea I - grile

1. LinkedHashMap cu cheie null

Răspuns pe scurt

Răspuns: {a=null, b=JavaSE, c=Python, null=null}.

LinkedHashMap păstrează ordinea inserării. Dacă se inserează din nou aceeași cheie, se schimbă valoarea, dar cheia nu se mută la final. Cheia null este permisă.

2. Ascunderea câmpurilor

Răspuns pe scurt

Output: 10 20.

A ob = new B(); are tip declarat A și tip real B. Pentru câmpuri, inclusiv statice, accesul se face după tipul declarat al referinței, deci ob.x este A.x, iar ob.y este A.y.

3. Excepții și finally

Răspuns pe scurt

Output: ABDEC.

Este aceeași logică precum la subiectul din 2022: B și D se adaugă înainte de excepție, E se adaugă în finally, iar C se adaugă în catch-ul metodei A().

4. Iterator și ConcurrentModificationException

Răspuns pe scurt

Adevărată: apelul remove pe listă, în timp ce parcurgem cu iteratorul, generează ConcurrentModificationException la rulare.

Trebuia folosit itr.remove(), nu numere.remove(nr). Modificarea directă a listei invalidează iteratorul.

5. Overriding, overloading, erori

Răspuns pe scurt

Răspuns: o suprascriere, o supraîncărcare, trei erori.

met1() este suprascriere validă. met2() este eroare deoarece în A este final. met3() este eroare prin reducerea accesului. met4() este eroare prin reducerea accesului la o metodă statică ascunsă. met5() este supraîncărcare.

3.2 Partea II - clasa Produs și stream-uri

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 class Produs {
5     private String firma;
6     private String produs;
7     private float cantitate;
8     private float pretUnitar;
9
10    public Produs(String firma, String produs, float cantitate, float pretUnitar) {
11        this.firma = firma;
12        this.produs = produs;
13        this.cantitate = cantitate;
14        this.pretUnitar = pretUnitar;
15    }
16
17    public String getFirma() { return firma; }
18    public String getProdus() { return produs; }
19    public float getCantitate() { return cantitate; }
20    public float getPretUnitar() { return pretUnitar; }
21
22    public void setFirma(String firma) { this.firma = firma; }
23    public void setProdus(String produs) { this.produs = produs; }
24    public void setCantitate(float cantitate) { this.cantitate = cantitate; }
25    public void setPretUnitar(float pretUnitar) { this.pretUnitar = pretUnitar; }
26
27    public float valoareTotala() {
28        return cantitate * pretUnitar;
29    }
30
31    @Override
32    public String toString() {
33        return firma + " - " + produs + ", cantitate=" + cantitate
34            + ", pret=" + pretUnitar;
35    }
36
37    @Override
38    public boolean equals(Object o) {
39        if (this == o) return true;
40        if (!(o instanceof Produs other)) return false;
41        return Float.compare(cantitate, other.cantitate) == 0
42            && Float.compare(pretUnitar, other.pretUnitar) == 0
43            && Objects.equals(firma, other.firma)
44            && Objects.equals(produs, other.produs);
45    }
46
47    @Override
48    public int hashCode() {
49        return Objects.hash(firma, produs, cantitate, pretUnitar);
50    }
51 }
52
53 public class TestProdus {
54     public static void main(String[] args) {
55         List<Produs> produse = new ArrayList<>(List.of(
56             new Produs("FirmaA", "lapte", 20, 7.5f),
57             new Produs("FirmaB", "telefon", 3, 1200f),
58             new Produs("FirmaA", "paine", 100, 3.5f),
59             new Produs("FirmaC", "monitor", 2, 800f)
60         ));
61
62         // a) produse cu pret unitar >= 100 RON, descrescator dupa cantitate
63         produse.stream()
64             .filter(p -> p.getPretUnitar() >= 100)
65             .sorted(Comparator.comparing(Produs::getCantitate).reversed())

```

```

66         .forEach(System.out::println);
67
68     // b) firme distincte, alfabetice
69     produse.stream()
70         .map(Produs::getFirma)
71         .distinct()
72         .sorted()
73         .forEach(System.out::println);
74
75     // c) colectie cu produse cu valoare totala cel mult 1000 RON
76     List<Produs> produseIeftine = produse.stream()
77         .filter(p -> p.valoareTotala() <= 1000)
78         .collect(Collectors.toList());
79
80     // d) pretul unitar mediu
81     double medie = produse.stream()
82         .mapToDouble(Produs::getPretUnitar)
83         .average()
84         .orElse(0);
85     System.out.println(medie);
86 }
87 }

```

3.3 Partea III - fișiere + fire pentru produse

```

1  import java.io.*;
2  import java.nio.file.*;
3  import java.util.*;
4
5  class ProdusParser {
6      static Produs parse(String linie) {
7          String[] v = linie.split(",");
8          return new Produs(
9              v[0].trim(),
10             v[1].trim(),
11             Float.parseFloat(v[2].trim()),
12             Float.parseFloat(v[3].trim())
13         );
14     }
15 }
16
17 class CalculatorValoareFirma extends Thread {
18     private final Path fisier;
19     private final String firma;
20     private float total;
21
22     public CalculatorValoareFirma(String fisier, String firma) {
23         this.fisier = Path.of(fisier);
24         this.firma = firma;
25     }
26
27     public float getTotal() { return total; }
28
29     @Override
30     public void run() {
31         try {
32             total = (float) Files.lines(fisier)
33                 .map(ProdusParser::parse)
34                 .filter(p -> p.getFirma().equalsIgnoreCase(firma))
35                 .mapToDouble(Produs::valoareTotala)
36                 .sum();
37         } catch (IOException e) {
38             throw new RuntimeException(e);
39         }
40     }

```

```

41 }
42
43 public class MainProduseFisiere {
44     public static void main(String[] args) throws Exception {
45         Scanner sc = new Scanner(System.in);
46         String firma = sc.nextLine();
47
48         CalculatorValoareFirma f1 = new CalculatorValoareFirma("magazin_1.txt", firma);
49         CalculatorValoareFirma f2 = new CalculatorValoareFirma("magazin_2.txt", firma);
50
51         f1.start();
52         f2.start();
53         f1.join();
54         f2.join();
55
56         System.out.println(f1.getTotal() + f2.getTotal());
57     }
58 }

```

3.4 Partea IV - clasă imutabilă Facultate

```

1 import java.util.*;
2
3 public final class Facultate {
4     private final String denumireFacultate;
5     private final int numarStudenti;
6     private final List<String> specializari;
7     private final Adresa adresa;
8
9     public Facultate(String denumireFacultate, int numarStudenti,
10         List<String> specializari, Adresa adresa) {
11         this.denumireFacultate = denumireFacultate;
12         this.numarStudenti = numarStudenti;
13         this.specializari = List.copyOf(specializari);
14         this.adresa = new Adresa(adresa);
15     }
16
17     public String getDenumireFacultate() { return denumireFacultate; }
18     public int getNumarStudenti() { return numarStudenti; }
19
20     public List<String> getSpecializari() {
21         return List.copyOf(specializari);
22     }
23
24     public Adresa getAdresa() {
25         return new Adresa(adresa);
26     }
27
28     @Override
29     public String toString() {
30         return denumireFacultate + " " + numarStudenti + " "
31             + specializari + " " + adresa;
32     }
33
34     @Override
35     public boolean equals(Object o) {
36         if (this == o) return true;
37         if (!(o instanceof Facultate other)) return false;
38         return numarStudenti == other.numarStudenti
39             && Objects.equals(denumireFacultate, other.denumireFacultate)
40             && Objects.equals(specializari, other.specializari)
41             && Objects.equals(adresa, other.adresa);
42     }
43
44     @Override

```

```
45 public int hashCode() {  
46     return Objects.hash(denumireFacultate, numarStudenti, specializari, adresa);  
47 }  
48 }
```

4 Subiect 01 - 2024, seria 25

4.1 Partea I - grile

1. Overriding, overloading, erori

Răspuns pe scurt

Răspuns: o suprascriere, o supraîncărcare, trei erori.

`met1()` este suprascriere validă. `met2()` încearcă să suprascrie o metodă final. `met3()` reduce accesul. `met4()` încearcă să ascundă static o metodă de instanță. `met5()` are altă listă de parametri, deci este supraîncărcare.

2. HashSet cu equals inversat

Răspuns pe scurt

Output: 4.

`equals()` întoarce `true` când valorile sunt diferite și `false` când sunt egale. Se verifică doar în același bucket:

- `A(100)`: hash 1, se adaugă;
- `A(10)`: hash 0, se adaugă;
- `A(1)`: hash 0, este considerat egal cu `A(10)` pentru că $1 \neq 10$, deci nu se adaugă;
- `A(10)`: hash 0, nu este egal cu `A(10)` existent pentru că $10 \neq 10$ este fals, deci se adaugă;
- `A(100)`: hash 1, nu este egal cu `A(100)` existent, deci se adaugă.

Total: 4 elemente.

3. Câmp ascuns și metodă suprascrisă

Răspuns pe scurt

Output: 130.

`ob.x` se ia după tipul referinței, deci `A.x = 10`. Metoda `f` este nestatică și suprascrisă, deci se execută `B.f`. În `B.f`, câmpul `x` este cel din `B`, adică 30. Rezultatul este $30 + 10 \cdot 10 = 130$.

4. Excepții și finally

Răspuns pe scurt

Output: EACDB.

Pornim de la E. Metoda `A()` adaugă A. Metoda `B()` adaugă C, apoi `C()` aruncă excepție. `catch` din `B()` relansează, `finally` adaugă D, iar `catch` din `A()` adaugă B.

5. Interfață funcțională și default method

Răspuns pe scurt

Sunt adevărate 4 afirmații: a), c), e), f).

- a) adevărată: Funcție are o singură metodă abstractă, `f(double)`.
- b) falsă: Test nu implementează Funcție, deci nu rescrie metoda default.
- c) adevărată: `x -> 10` este compatibilă cu `int f(double)`.
- d) falsă: `x -> x / 100` întoarce `double`, dar metoda trebuie să întoarcă `int`.
- e) adevărată: clasa anonimă implementează metoda abstractă `f`.
- f) adevărată: deoarece clasa anonimă nu suprascrie `eval`, se apelează metoda default din interfață.

4.2 Partea II - clasa TennisATP și stream-uri

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 class TennisATP {
5     private String numeJucator;
6     private String nationalitateJucator;
7     private int pozitieATP;
8     private int puncteATP;
9     private int nrTurneeATP;
10
11     public TennisATP(String numeJucator, String nationalitateJucator,
12         int pozitieATP, int puncteATP, int nrTurneeATP) {
13         this.numeJucator = numeJucator;
14         this.nationalitateJucator = nationalitateJucator;
15         this.pozitieATP = pozitieATP;
16         this.puncteATP = puncteATP;
17         this.nrTurneeATP = nrTurneeATP;
18     }
19
20     public String getNumeJucator() { return numeJucator; }
21     public String getNationalitateJucator() { return nationalitateJucator; }
22     public int getPozitieATP() { return pozitieATP; }
23     public int getPuncteATP() { return puncteATP; }
24     public int getNrTurneeATP() { return nrTurneeATP; }
25
26     @Override
27     public String toString() {
28         return numeJucator + " (" + nationalitateJucator + ") "
29             + pozitieATP + ", " + puncteATP + " pct, "
30             + nrTurneeATP + " turnee";
31     }
32
33     @Override
34     public boolean equals(Object o) {
35         if (this == o) return true;
36         if (!(o instanceof TennisATP other)) return false;
37         return pozitieATP == other.pozitieATP
38             && puncteATP == other.puncteATP
39             && nrTurneeATP == other.nrTurneeATP
40             && Objects.equals(numeJucator, other.numeJucator)
41             && Objects.equals(nationalitateJucator, other.nationalitateJucator);
42     }
43

```

```

44  @Override
45  public int hashCode() {
46      return Objects.hash(numeJucator, nationalitateJucator,
47                          pozitieATP, puncteATP, nrTurneeATP);
48  }
49  }
50
51  public class TestTennisATP {
52      public static void main(String[] args) {
53          List<TennisATP> jucatori = new ArrayList<>(List.of(
54              new TennisATP("Popescu", "Romania", 110, 900, 8),
55              new TennisATP("Alcaraz", "Spania", 2, 8000, 18),
56              new TennisATP("Rune", "Danemarca", 15, 2500, 12),
57              new TennisATP("Musetti", "Italia", 20, 1800, 11)
58          ));
59
60          // a) cel puțin 10 turnee, sortati alfabetic dupa nume
61          jucatori.stream()
62              .filter(j -> j.getNrTurneeATP() >= 10)
63              .sorted(Comparator.comparing(TennisATP::getNumJucator))
64              .forEach(System.out::println);
65
66          // b) nationalitati distincte
67          jucatori.stream()
68              .map(TennisATP::getNationalitateJucator)
69              .distinct()
70              .forEach(System.out::println);
71
72          // c) colectie cu jucatori intre 1000 si 4000 puncte
73          List<TennisATP> punctajMediu = jucatori.stream()
74              .filter(j -> j.getPuncteATP() >= 1000 && j.getPuncteATP() <= 4000)
75              .collect(Collectors.toList());
76
77          // d) numarul de jucatori pentru fiecare nationalitate
78          jucatori.stream()
79              .collect(Collectors.groupingBy(
80                  TennisATP::getNationalitateJucator,
81                  Collectors.counting()))
82              .forEach((nat, nr) -> System.out.println(nat + " -> " + nr));
83      }
84  }

```

4.3 Partea III - fișiere + fire pentru jucători

```

1  import java.io.*;
2  import java.nio.file.*;
3  import java.util.*;
4
5  class TennisParser {
6      static TennisATP parse(String linie) {
7          String[] v = linie.split(",");
8          return new TennisATP(
9              v[0].trim(),
10             v[1].trim(),
11             Integer.parseInt(v[2].trim()),
12             Integer.parseInt(v[3].trim()),
13             Integer.parseInt(v[4].trim())
14         );
15     }
16 }
17
18 class NumaratorNationalitate extends Thread {
19     private final Path fisier;
20     private final String nationalitate;
21     private int rezultat;

```

```
22
23 public NumaratorNationalitate(String fisier, String nationalitate) {
24     this.fisier = Path.of(fisier);
25     this.nationalitate = nationalitate;
26 }
27
28 public int getRezultat() { return rezultat; }
29
30 @Override
31 public void run() {
32     try {
33         rezultat = (int) Files.lines(fisier)
34             .map(TenisParser::parse)
35             .filter(j -> j.getNationalitateJucator().equalsIgnoreCase(nationalitate))
36             .count();
37     } catch (IOException e) {
38         throw new RuntimeException(e);
39     }
40 }
41 }
42
43 public class MainJucatori {
44     public static void main(String[] args) throws Exception {
45         Scanner sc = new Scanner(System.in);
46         String nationalitate = sc.nextLine();
47
48         NumaratorNationalitate f1 = new NumaratorNationalitate("jucatori_1.txt",
49             nationalitate);
50         NumaratorNationalitate f2 = new NumaratorNationalitate("jucatori_2.txt",
51             nationalitate);
52
53         f1.start();
54         f2.start();
55         f1.join();
56         f2.join();
57
58         System.out.println(f1.getRezultat() + f2.getRezultat());
59     }
60 }
```

4.4 Partea IV - Facultate imutabilă

Se folosește aceeași soluție ca la secțiunea 3.4: String se poate atribui direct, int se copiază prin valoare, List<String> se copiază cu List.copyOf, iar Adresa trebuie copiată defensiv în constructor și în getter.

5 Subiect 02 - 2024, seriile 23 și 24

5.1 Partea I - grile

1. Fișiere goale și InputMismatchException

Răspuns pe scurt

Cu codul exact din poză, programul afișează doar F și se termină cu `NoSuchElementException`. Niciuna dintre variantele listate nu corespunde exact.

De ce? Pentru `file1.txt` gol, `Scanner.nextInt()` aruncă `NoSuchElementException`, nu `InputMismatchException`. Cum acest tip nu este prins, se execută doar `finally`, deci se afișează F, apoi programul se oprește cu excepție.

Atenție

Dacă în subiect ar fi fost `catch(NoSuchElementException e)` în loc de `catch(InputMismatchException e)`, atunci răspunsul ar fi fost ?F2F?F4F!F cu terminare prin excepție, ca în varianta clasică a subiectului.

2. Clase imutabile A, B, C

Răspuns pe scurt

Răspuns: 0 clase imutabile, dacă analizăm riguros codul exact, deoarece câmpurile nu sunt `private final`, A are setter, iar B/C expun sau copiază superficial obiecte mutabile.

Pentru o clasă imutabilă corectă, câmpurile trebuie să fie `private`, `finale` și obiectele mutabile trebuie copiate defensiv.

3. LinkedHashSet, substring și operatorul ==

Răspuns pe scurt

Output: examen Java Examen PA0 Java.

`hashCode()` folosește primul caracter. Pentru cele două obiecte "Java", hash-ul este același, dar `equals()` compară cu `==` două rezultate `substring(1)`, adică referințe diferite. De aceea al doilea "Java" se adaugă. `LinkedHashSet` păstrează ordinea inserării.

4. Constructor, câmp ascuns, super.f

Răspuns pe scurt

Output: 340.

`A ob = new B();` face apel la `B()`, care apelează `super(20)`, deci `A.x = 20`. Expresia `ob.x` folosește câmpul din A, deci parametrul este 20. Se execută `B.f(20)`:

$$20 + \text{super.f}(10 \cdot B.x) = 20 + \text{super.f}(300) = 20 + (20 + 300) = 340.$$

5. Interfață nefuncțională și erori de compilare

Răspuns pe scurt

Răspuns: 3 erori de compilare.

- În metoda afisare, expresia `f.f(t)` este eroare: tipul declarat `Funcție` nu are metoda `f(int)`; are doar un câmp static `f` de tip `Function<Integer, Double>`.
- `afisare(new A().f, 20)` este eroare: `new A().f` se referă la câmpul `Function<Integer, Double>`, nu la un obiect de tip `Funcție`.
- `afisare(x -> 3.0 / x, 20)` este eroare: `Funcție` nu este interfață funcțională, deoarece nu are nicio metodă abstractă.

5.2 Partea II - Imobil și stream-uri

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 class Imobil {
5     private String tip;
6     private String localitate;
7     private int nrCamere;
8     private double suprafata;
9     private double pret;
10
11     public Imobil(String tip, String localitate, int nrCamere,
12                  double suprafata, double pret) {
13         this.tip = tip;
14         this.localitate = localitate;
15         this.nrCamere = nrCamere;
16         this.suprafata = suprafata;
17         this.pret = pret;
18     }
19
20     public String getTip() { return tip; }
21     public String getLocalitate() { return localitate; }
22     public int getNrCamere() { return nrCamere; }
23     public double getSuprafata() { return suprafata; }
24     public double getPret() { return pret; }
25
26     @Override
27     public String toString() {
28         return tip + ", " + localitate + ", " + nrCamere
29             + ", " + suprafata + ", " + pret;
30     }
31
32     @Override
33     public boolean equals(Object o) {
34         if (this == o) return true;
35         if (!(o instanceof Imobil other)) return false;
36         return nrCamere == other.nrCamere
37             && Double.compare(suprafata, other.suprafata) == 0
38             && Double.compare(pret, other.pret) == 0
39             && Objects.equals(tip, other.tip)
40             && Objects.equals(localitate, other.localitate);
41     }
42
43     @Override
44     public int hashCode() {
45         return Objects.hash(tip, localitate, nrCamere, suprafata, pret);
46     }

```

```

47 }
48
49 public class TestImobill {
50     public static void main(String[] args) {
51         List<Imobill> imobile = new ArrayList<>(List.of(
52             new Imobill("apartament", "Bucuresti", 3, 70, 420000),
53             new Imobill("casa", "Bucuresti", 5, 140, 800000),
54             new Imobill("apartament", "Cluj", 3, 68, 490000),
55             new Imobill("garsoniera", "Iasi", 1, 34, 60000)
56         ));
57
58         // a) cel putin 3 camere, pret <= 500000, crescator dupa suprafata
59         imobile.stream()
60             .filter(i -> i.getNrCamere() >= 3)
61             .filter(i -> i.getPret() <= 500000)
62             .sorted(Comparator.comparingDouble(Imobill::getSuprafata))
63             .forEach(System.out::println);
64
65         // b) localitati distincte
66         imobile.stream()
67             .map(Imobill::getLocalitate)
68             .distinct()
69             .forEach(System.out::println);
70
71         // c) Bucuresti, pret intre 300000 si 500000
72         List<Imobill> rezultat = imobile.stream()
73             .filter(i -> i.getLocalitate().equalsIgnoreCase("Bucuresti"))
74             .filter(i -> i.getPret() >= 300000 && i.getPret() <= 500000)
75             .collect(Collectors.toList());
76
77         // d) pentru fiecare localitate distincta, imobilele din localitatea respectiva
78         imobile.stream()
79             .collect(Collectors.groupingBy(Imobill::getLocalitate))
80             .forEach((loc, lista) -> System.out.println(loc + " -> " + lista));
81     }
82 }

```

5.3 Partea III - fișiere + fire pentru AgentieRS

```

1 import java.io.*;
2 import java.nio.file.*;
3 import java.util.*;
4
5 class ImobillParser {
6     static Imobill parse(String linie) {
7         String[] v = linie.split(",");
8         return new Imobill(v[0].trim(), v[1].trim(),
9             Integer.parseInt(v[2].trim()),
10             Double.parseDouble(v[3].trim()),
11             Double.parseDouble(v[4].trim()));
12     }
13 }
14
15 class NumaratorAgentieRS extends Thread {
16     private final Path fisier;
17     private final double smin;
18     private final int cmin;
19     private int rezultat;
20
21     public NumaratorAgentieRS(String fisier, double smin, int cmin) {
22         this.fisier = Path.of(fisier);
23         this.smin = smin;
24         this.cmin = cmin;
25     }
26 }

```

```

27 public int getRezultat() { return rezultat; }
28
29 @Override
30 public void run() {
31     try {
32         rezultat = (int) Files.lines(fisier)
33             .map(ImobiliuParser::parse)
34             .filter(i -> i.getSuprafata() > smin)
35             .filter(i -> i.getNrCamere() >= cmin)
36             .count();
37     } catch (IOException e) {
38         throw new RuntimeException(e);
39     }
40 }
41 }
42
43 public class MainAgentieRS {
44     public static void main(String[] args) throws Exception {
45         Scanner sc = new Scanner(System.in);
46         double smin = sc.nextDouble();
47         int cmin = sc.nextInt();
48
49         NumaratorAgentieRS t1 = new NumaratorAgentieRS("AgentieRS_1.txt", smin, cmin);
50         NumaratorAgentieRS t2 = new NumaratorAgentieRS("AgentieRS_2.txt", smin, cmin);
51
52         t1.start();
53         t2.start();
54         t1.join();
55         t2.join();
56
57         System.out.println(t1.getRezultat() + t2.getRezultat());
58     }
59 }

```

5.4 Partea IV - singleton CitireInformatiiPersoane

```

1 import java.io.*;
2 import java.nio.file.*;
3 import java.util.*;
4
5 class Persoana {
6     private String nume;
7     private int varsta;
8     private double salariuMediuAnual;
9
10    public Persoana(String nume, int varsta, double salariuMediuAnual) {
11        this.nume = nume;
12        this.varsta = varsta;
13        this.salariuMediuAnual = salariuMediuAnual;
14    }
15
16    public String getNume() { return nume; }
17    public int getVarsta() { return varsta; }
18    public double getSalariuMediuAnual() { return salariuMediuAnual; }
19
20    @Override
21    public String toString() {
22        return nume + ", " + varsta + ", " + salariuMediuAnual;
23    }
24 }
25
26 public final class CitireInformatiiPersoane {
27     private static CitireInformatiiPersoane instanta;
28
29     private CitireInformatiiPersoane() { }

```

```
30
31 public static CitireInformatiiPersoane getInstanta() {
32     if (instanta == null) {
33         instanta = new CitireInformatiiPersoane();
34     }
35     return instanta;
36 }
37
38 public ArrayList<Persoana> citesteDinCsv(String numeFisier) throws IOException {
39     ArrayList<Persoana> persoane = new ArrayList<>();
40
41     try (BufferedReader br = Files.newBufferedReader(Path.of(numeFisier))) {
42         String linie;
43         while ((linie = br.readLine()) != null) {
44             String[] v = linie.split(",");
45             Persoana p = new Persoana(
46                 v[0].trim(),
47                 Integer.parseInt(v[1].trim()),
48                 Double.parseDouble(v[2].trim())
49             );
50             persoane.add(p);
51         }
52     }
53
54     return persoane;
55 }
56 }
```

6 Examen 2024-2025

6.1 Partea I - grile

1. HashSet cu hashCode și equals neobișnuite

Răspuns pe scurt

Output: 3.

`hashCode()` este `marca.length() / 2`. `equals()` întoarce `true` când prima literă este diferită, adică invers față de logica normală. Elemente:

- Dacia: hash 2, se adaugă;
- Bugatti: hash 3, se adaugă;
- Ford: hash 2, este comparat cu Dacia; primele litere diferă, deci este considerat egal și nu se adaugă;
- Borgward: hash 4, se adaugă;
- Fiat: hash 2, considerat egal cu Dacia, nu se adaugă;
- Delage: hash 3, considerat egal cu Bugatti, nu se adaugă.

Total: 3.

2. Referințe, overload și obiecte String

Răspuns pe scurt

Output: Matei Ion.

`s.modificaSir("Matei")` modifică obiectul `s`. Apelul `t.modificaSir(new Sir("Dan"))` intră în metoda care primește un `Sir`; acolo se modifică doar parametrul local `sir`, nu câmpul lui `t`. La fel, `s.modificaSir(t)` nu schimbă `s`. Deci `s` rămâne Matei, iar `t` rămâne Ion.

3. Scanner, InputMismatchException și finally

Răspuns pe scurt

Răspuns: se afișează o singură dată mesajul Număr incorect! și suma numerelor citite înaintea primului token incorect; programul se termină normal.

La primul token care nu poate fi citit ca `int`, `nextInt()` aruncă `InputMismatchException`. Se iese din `while`, se execută `catch`, apoi `finally`, unde se afișează suma acumulată. Programul nu revine în `while`.

4. Polimorfism pe metode

Răspuns pe scurt

Output: A B B.

`p` este obiect real `B`; clasa `B` nu suprascrie metoda, deci afișează `A`. `q` și `r` sunt obiecte reale `C`; metoda `print()` este suprascrisă în `C`, deci afișează `B`.

5. Eliminare perechi de paranteze

Răspuns pe scurt

Se afișează numărul minim de paranteze care trebuie inserate ca șirul să devină corect parantezat.

Algoritmul elimină repetat perechi adiacente (). Prin eliminări repetate dispar toate perechile care se închid corect. Ce rămâne sunt parantezele neasociate; numărul lor este numărul minim de paranteze care trebuie adăugate.

6.2 Partea II - clasa Imobil și stream-uri

În acest subiect, clasa are câmpurile tipImobil, nrCamere, suprafata, localitate; nu are câmp pret.

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 class Imobil2025 {
5     private String tipImobil;
6     private int nrCamere;
7     private double suprafata;
8     private String localitate;
9
10    public Imobil2025(String tipImobil, int nrCamere,
11                      double suprafata, String localitate) {
12        this.tipImobil = tipImobil;
13        this.nrCamere = nrCamere;
14        this.suprafata = suprafata;
15        this.localitate = localitate;
16    }
17
18    public String getTipImobil() { return tipImobil; }
19    public int getNrCamere() { return nrCamere; }
20    public double getSuprafata() { return suprafata; }
21    public String getLocalitate() { return localitate; }
22
23    @Override
24    public String toString() {
25        return tipImobil + ", " + nrCamere + " camere, "
26            + suprafata + " mp, " + localitate;
27    }
28
29    @Override
30    public boolean equals(Object o) {
31        if (this == o) return true;
32        if (!(o instanceof Imobil2025 other)) return false;
33        return nrCamere == other.nrCamere
34            && Double.compare(suprafata, other.suprafata) == 0
35            && Objects.equals(tipImobil, other.tipImobil)
36            && Objects.equals(localitate, other.localitate);
37    }
38
39    @Override
40    public int hashCode() {
41        return Objects.hash(tipImobil, nrCamere, suprafata, localitate);
42    }
43 }
44
45 public class TestImobil2025 {
46     public static void main(String[] args) {
47         List<Imobil2025> imobile = new ArrayList<>(List.of(
48             new Imobil2025("apartament", 2, 55, "Bucuresti"),

```

```

49     new Imobil2025("casa", 5, 130, "Bucuresti"),
50     new Imobil2025("garsoniera", 1, 32, "Cluj"),
51     new Imobil2025("apartament", 2, 75, "Iasi")
52 );
53
54 // a) imobile din Bucuresti, sortate crescator dupa nrCamere
55 imobile.stream()
56     .filter(i -> i.getLocalitate().equalsIgnoreCase("Bucuresti"))
57     .sorted(Comparator.comparingInt(Imobil2025::getNrCamere))
58     .forEach(System.out::println);
59
60 // b) localitati distincte
61 imobile.stream()
62     .map(Imobil2025::getLocalitate)
63     .distinct()
64     .forEach(System.out::println);
65
66 // c) lista cu suprafata intre 50 si 100 mp
67 List<Imobil2025> suprafeteMedii = imobile.stream()
68     .filter(i -> i.getSuprafata() >= 50 && i.getSuprafata() <= 100)
69     .collect(Collectors.toList());
70
71 // d) numarul de imobile cu doua camere din fiecare localitate
72 imobile.stream()
73     .filter(i -> i.getNrCamere() == 2)
74     .collect(Collectors.groupingBy(
75         Imobil2025::getLocalitate,
76         Collectors.counting()))
77     .forEach((loc, nr) -> System.out.println(loc + " -> " + nr));
78 }
79 }

```

6.3 Partea III - fișiere + fire pentru localitate

```

1  import java.io.*;
2  import java.nio.file.*;
3  import java.util.*;
4
5  class Imobil2025Parser {
6      static Imobil2025 parse(String linie) {
7          String[] v = linie.split(",");
8          return new Imobil2025(
9              v[0].trim(),
10             Integer.parseInt(v[1].trim()),
11             Double.parseDouble(v[2].trim()),
12             v[3].trim()
13         );
14     }
15 }
16
17 class NumaratorLocalitate extends Thread {
18     private final Path fisier;
19     private final String localitate;
20     private int rezultat;
21
22     public NumaratorLocalitate(String fisier, String localitate) {
23         this.fisier = Path.of(fisier);
24         this.localitate = localitate;
25     }
26
27     public int getRezultat() { return rezultat; }
28
29     @Override
30     public void run() {
31         try {

```

```

32         rezultat = (int) Files.lines(fisier)
33             .map(Imobil2025Parser::parse)
34             .filter(i -> i.getLocalitate().equalsIgnoreCase(localitate))
35             .count();
36     } catch (IOException e) {
37         throw new RuntimeException(e);
38     }
39 }
40 }
41
42 public class MainImobile2025 {
43     public static void main(String[] args) throws Exception {
44         Scanner sc = new Scanner(System.in);
45         String localitate = sc.nextLine();
46
47         NumaratorLocalitate t1 = new NumaratorLocalitate("imobile_1.txt", localitate);
48         NumaratorLocalitate t2 = new NumaratorLocalitate("imobile_2.txt", localitate);
49
50         t1.start();
51         t2.start();
52         t1.join();
53         t2.join();
54
55         System.out.println(t1.getRezultat() + t2.getRezultat());
56     }
57 }

```

6.4 Partea IV - clasă imutabilă Imobil cu StringBuilder

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 public final class ImobilImutabil {
5     private final Adresa adresa;
6     private final int numarLocatari;
7     private final List<StringBuilder> locatari;
8
9     public ImobilImutabil(Adresa adresa, int numarLocatari,
10         List<StringBuilder> locatari) {
11         this.adresa = new Adresa(adresa);
12         this.numarLocatari = numarLocatari;
13         this.locatari = locatari.stream()
14             .map(sb -> new StringBuilder(sb.toString()))
15             .collect(Collectors.toUnmodifiableList());
16     }
17
18     public Adresa getAdresa() {
19         return new Adresa(adresa);
20     }
21
22     public int getNumarLocatari() {
23         return numarLocatari;
24     }
25
26     public List<StringBuilder> getLocatari() {
27         return locatari.stream()
28             .map(sb -> new StringBuilder(sb.toString()))
29             .collect(Collectors.toList());
30     }
31
32     private List<String> locatariCaStringuri() {
33         return locatari.stream()
34             .map(StringBuilder::toString)
35             .collect(Collectors.toList());
36     }

```

```
37
38
39 @Override
40 public String toString() {
41     return adresa + ", " + numarLocatari + ", " + locatariCaStringuri();
42 }
43
44 @Override
45 public boolean equals(Object o) {
46     if (this == o) return true;
47     if (!(o instanceof ImobilImutabil other)) return false;
48     return numarLocatari == other.numarLocatari
49         && Objects.equals(adresa, other.adresa)
50         && Objects.equals(locatariCaStringuri(), other.locatariCaStringuri());
51 }
52
53 @Override
54 public int hashCode() {
55     return Objects.hash(adresa, numarLocatari, locatariCaStringuri());
56 }
```

Regulă de examen

Pentru `List<StringBuilder>`, `List.copyOf(lista)` nu este suficient. Aceasta copiază lista, dar nu copiază obiectele `StringBuilder` din interior. Trebuie făcută copie profundă: fiecare `StringBuilder` devine new `StringBuilder(sb.toString())`.

7 Tabel rapid de reguli pentru examen

Concept	Regulă scurtă
String	Este imutabil. Metodele ca <code>toUpperCase()</code> nu modifică obiectul, ci întorc un șir nou.
<code>substring(a,b)</code> <code>==</code> pe obiecte	Include indexul <code>a</code> , exclude indexul <code>b</code> . Compară referințe, nu conținut. Pentru conținut se folosește <code>equals()</code> .
HashSet / HashMap	Întâi bucket după <code>hashCode()</code> , apoi <code>equals()</code> doar în același bucket.
Câmpuri ascunse	Câmpul accesat depinde de tipul referinței, nu de tipul real al obiectului.
Metode suprascrise	Implementarea metodei nestatice se alege după tipul real al obiectului.
<code>final method</code>	Nu poate fi suprascrisă.
<code>static method</code>	Se ascunde, nu se suprascrie polimorfic.
Acces la overriding	Accesul nu poate deveni mai restrictiv.
<code>finally</code>	Se execută și dacă apare excepție, și dacă există <code>return</code> , și înainte de propagarea excepției.
<code>start()</code> vs <code>run()</code>	<code>start()</code> creează fir nou; <code>run()</code> este apel normal de metodă.
Clasă imutabilă	<code>final class</code> , câmpuri private <code>final</code> , fără setteri, copii defensive pentru obiecte mutabile.
Stream	Operațiile intermediare sunt lazy; se execută doar la operație terminală. Stream-ul nu se reutilizează.